

System Design for Large-Scale Reinforcement Learning Workloads

WANG, Chujiao

Student ID: 224045007

A Thesis Submitted in

CSC6032 Advanced Operating System

School of Data Science

The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Modern reinforcement learning is no longer constrained only by learning algorithms. Its progress also depends on the systems that generate experience, run policy inference, move trajectories, and update neural networks. A large RL workload combines CPU-heavy simulation, accelerator-friendly inference and training, frequent communication, and continuous producer-consumer coordination. These properties make the topic a natural fit for an Advanced Operating System course: learning performance is shaped by scheduling, synchronization, batching, memory sharing, remote procedure calls, load balance, and the placement of computation across heterogeneous resources.

This thesis-style report studies three representative systems: IMPALA, SEED RL, and Sample Factory. IMPALA introduced an importance-weighted actor-learner architecture that decouples acting from learning and uses V-trace to correct policy lag. SEED RL reorganized the actor-learner model by moving inference to centralized accelerators and by using streaming communication to reduce CPU inference cost and improve accelerator utilization. Sample Factory showed that careful single-machine engineering can reach high throughput through asynchronous rollout, policy inference, learning, shared memory, and double-buffered sampling. Although these papers appear in the reinforcement learning literature, their central contributions are systems contributions. Each one changes where work runs, when components wait, what data crosses component boundaries, and how much stale computation the algorithm can tolerate.

The report follows the logic of the course presentation. It first frames reinforcement learning as a systems workload, then analyzes bottlenecks in conventional training, then reviews the three systems in detail, and finally compares their operating-system implications. The central argument is that scalable reinforcement learning is

an exercise in algorithm-system co-design. High throughput comes from reducing idle time, placing computation on suitable resources, and choosing learning algorithms that remain stable under asynchronous execution.

Keywords: reinforcement learning systems, distributed training, actor-learner architecture, asynchronous execution, V-trace, centralized inference, shared memory, GPU utilization, operating systems.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Why This Topic Belongs to Advanced Operating System	2
1.3 Report Scope and Contributions	3
1.4 Organization	3
2 Reinforcement Learning as a Systems Workload	4
2.1 The Training Loop	4
2.2 Major Components	5
2.3 Closed-Loop Data Generation	5
2.4 Throughput and Wall-Clock Learning	6
2.5 Baseline Bottlenecks and Design Questions	7
2.5.1 Synchronous Training Bottlenecks	7
2.5.2 Actor Inference Bottlenecks	8
2.5.3 Pipeline Imbalance	8
2.5.4 Policy Lag	9
2.5.5 Communication and Data Movement	9
2.6 Workload Model and Operating-System Interface	9
3 IMPALA: Scalable Distributed Actor-Learner Training	13
3.1 Motivation	13
3.2 Architecture	13
3.3 Systems Interpretation	14

3.4	Queueing Behavior and Backpressure	15
3.5	V-trace	16
3.6	Performance Results	17
3.7	Strengths	17
3.8	Limitations	18
4	SEED RL: Accelerated Central Inference	19
4.1	Motivation	19
4.2	Architecture	19
4.3	Computation Placement	20
4.4	RPC Path and Batching Semantics	21
4.5	Policy Lag in SEED RL	22
4.6	Cost and Throughput	23
4.7	Strengths	23
4.8	Limitations	24
5	Sample Factory: Single-Machine High-Throughput RL	25
5.1	Motivation	25
5.2	Architecture	25
5.3	Asynchronous Pipeline	26
5.4	Double-Buffered Sampling	27
5.5	Shared Memory and Queues	27
5.6	Memory Ownership and Local Runtime Design	28
5.7	Policy Lag and Learning Stability	29
5.8	Performance Results	30
5.9	Strengths and Limitations	30
6	Broader Related Work in Reinforcement Learning Systems	31
6.1	A3C and Early Asynchronous Reinforcement Learning	31
6.2	Gorila, Ape-X, and Distributed Replay	32
6.3	R2D2 and Recurrent State Staleness	33

6.4	RLlib and Acme as Systems Frameworks	33
6.5	DD-PPO and Synchronous Decentralized Scaling	34
6.6	EnvPool and Environment Execution Engines	34
6.7	Positioning the Three Main Papers	35
7	Synthesis and Operating-System Discussion	37
7.1	High-Level Comparison	37
7.2	Computation Placement	37
7.3	Synchronization and Asynchrony	38
7.4	Communication Patterns	38
7.5	Relationship Between Throughput and Learning	39
7.6	A Unified View	39
7.7	Common Design Patterns	40
7.7.1	Pattern 1: Decouple Data Production from Model Update . .	40
7.7.2	Pattern 2: Treat Inference as a Shared Service	41
7.7.3	Pattern 3: Hide Latency through Overlap	42
7.7.4	Pattern 4: Keep Data Movement Explicit	42
7.8	Evaluation Methodology for RL Systems	43
7.8.1	Frames per Second Is Necessary but Not Sufficient	43
7.8.2	Utilization and Bottleneck Breakdown	44
7.8.3	Freshness and Policy-Lag Metrics	45
7.8.4	Reproducibility and Configuration Sensitivity	45
7.9	Engineering Guidelines from the Literature	46
7.10	Operating-System Concepts in the Three Papers	47
7.10.1	Scheduling	47
7.10.2	Synchronization	47
7.10.3	Batching	48
7.10.4	Shared Memory	48
7.10.5	Remote Procedure Calls	49
7.10.6	Load Balancing and Stragglers	49

7.10.7 Heterogeneous Resource Management	49
7.11 Critical Discussion	50
7.11.1 Why the Three Works Are Closely Related	50
7.11.2 Algorithm-System Co-Design	50
7.11.3 Cost and Accessibility	51
7.11.4 Remaining Challenges	51
7.11.5 Possible Future Directions	51
8 Conclusion	53

List of Figures

2.1	The reinforcement learning training loop as a systems pipeline. The loop is closed because the learner changes the policy that controls future data generation. This schematic is a synthesis of the workload structures discussed in [2, 3, 11].	6
3.1	IMPALA decouples actors and learner. Actors produce trajectories asynchronously, while the learner consumes batches for GPU-based updates. The schematic is redrawn from the architecture described in [2].	14
4.1	SEED RL centralizes model computation. Actors run environments and communicate observations/actions, while inference and training stay on the learner side. The schematic is redrawn from the centralized inference design in [3].	20
5.1	Sample Factory uses a single-machine asynchronous pipeline with roll-out workers, policy workers, shared memory, and a learner. The schematic is redrawn from the architecture described in [11].	26
5.2	Double-buffered sampling hides inference latency by alternating environment groups, following the sampling strategy proposed in [11]. . .	27

List of Tables

2.1	Core design questions in large-scale reinforcement learning systems.	7
-----	--	---

1 Introduction

1.1 Background

Reinforcement learning trains an agent through repeated interaction with an environment. At each time step, the agent observes a state, chooses an action, receives a reward, and moves to the next state. The objective is to learn a policy that maximizes expected long-term reward. Early deep RL successes such as DQN already showed that large amounts of environment interaction are central to performance [8]. In a simple textbook description, this loop looks compact and algorithmic. In a modern deep reinforcement learning system, however, the loop expands into a complex distributed workload. The system must run many environments in parallel, execute neural network inference for many observations, collect trajectories, store recurrent states, update model parameters, and distribute the newest policy to the components that need it.

This gap between the compact algorithm and the deployed system motivates the report. In supervised learning, the training data are usually fixed: the system reads mini-batches from storage or memory and repeatedly performs forward and backward passes. Reinforcement learning folds data generation into the training loop. Environments must be stepped millions or billions of times, and the data distribution changes as the policy changes. Slow data generation slows learning. Slow inference leaves environments waiting. Slow communication leaves actors or learners idle. Excessive asynchrony can push the learner toward stale trajectories and reduce sample efficiency.

The three main papers address these problems from complementary angles. IMPALA studies distributed actor-learner scaling and introduces V-trace to correct policy lag. SEED RL focuses on centralized inference and accelerator utilization. Sam-

ple Factory studies high-throughput single-machine training through asynchronous pipeline design. Together, they show that progress in RL systems depends less on blindly adding machines than on designing the execution structure carefully.

1.2 Why This Topic Belongs to Advanced Operating System

At first glance, reinforcement learning may look like a machine learning topic rather than an operating-system topic. The selected papers, however, are deeply tied to systems design. They concern process roles, queues, batching, asynchronous execution, shared memory, GPU scheduling, remote procedure calls, and resource utilization. These are classic operating-system and distributed-system concerns.

The actor-learner architecture resembles a producer-consumer system. Actors produce experience, learners consume it, and queues decouple their rates. The performance of this architecture depends on how much synchronization is required, whether slow actors block faster ones, how batches are formed, and how data are moved. Similarly, centralized inference in SEED RL is a computation placement problem. It asks whether inference should run near the environment on CPU actors or near the learner on accelerators. The answer depends on communication latency, batching efficiency, bandwidth, and accelerator utilization. Sample Factory then brings these issues to a single node, where shared memory and fine-grained pipeline overlap replace large-scale networked distribution.

The topic therefore serves as a bridge from artificial intelligence to computer systems. The papers are notable reinforcement learning works, but their key ideas are best understood through operating-system principles. This report treats large-scale reinforcement learning as a heterogeneous, closed-loop, latency-sensitive, and throughput-hungry systems workload.

1.3 Report Scope and Contributions

The report is a literature review of three works:

- **IMPALA:** *IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures*, published at ICML 2018 [2].
- **SEED RL:** *SEED RL: Scalable and Efficient Deep-RL with Accelerated Central Inference*, published at ICLR 2020 [3].
- **Sample Factory:** *Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning*, published at ICML 2020 [11].

The report offers a systems-oriented synthesis rather than a purely algorithmic summary. It identifies the common problem behind the three papers: how to organize environment simulation, inference, and learning so that hardware is used efficiently while the learning algorithm remains stable. It also compares the systems across computation placement, synchronization model, communication pattern, memory strategy, policy-lag handling, and scalability target.

1.4 Organization

Chapter 2 describes reinforcement learning as a computer-system workload and explains the baseline bottlenecks that motivate the reviewed systems. Chapters 3, 4, and 5 review IMPALA, SEED RL, and Sample Factory respectively. Chapter 6 broadens the literature review to related reinforcement learning systems such as A3C, Ape-X, R2D2, RLlib, Acme, DD-PPO, and EnvPool. Chapter 7 synthesizes the systems from an operating-system perspective and discusses design trade-offs, evaluation methods, and open challenges. Chapter 8 concludes the report.

2 Reinforcement Learning as a Systems Workload

2.1 The Training Loop

A reinforcement learning system repeatedly executes the following loop. The environment produces an observation. The policy network maps the observation to an action distribution or action value. The action is applied to the environment. The environment returns a reward and a next observation. The system stores these transitions and later uses them to update the policy. In a deep reinforcement learning setting, both inference and training are neural-network computations.

This loop can be described with the notation of Markov decision processes. Let x_t be the state or observation at time t , a_t the selected action, r_t the reward, and $\pi(a_t|x_t)$ the policy. The agent seeks to maximize the expected discounted return

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right],$$

where γ is a discount factor. This mathematical objective hides a systems reality: each term in the sum requires environment interaction, and the interactions must be generated at scale.

In modern benchmarks such as Atari, DeepMind Lab, VizDoom, and Google Research Football, a serious experiment may require hundreds of millions or even billions of frames. A single environment instance cannot generate data at that scale quickly enough, so systems run many environments in parallel. The resulting workload contains many independent simulation streams and one or more learners that aggregate their experience.

2.2 Major Components

The training system can be decomposed into four components.

Environment execution runs simulators. It is often CPU-bound, may involve rendering, physics, game logic, or reset logic, and can show high variance across episodes. This stage produces observations and rewards.

Policy inference takes observations and computes actions. It is a forward pass through a neural network. For small models this may run on CPUs, but for larger models it is more efficient on GPUs or TPUs with large batches.

Trajectory transport and storage collects sequences of observations, actions, rewards, value estimates, logits, and recurrent states. Depending on the system, this data may move through queues, shared memory, replay buffers, or remote calls.

Learning performs gradient updates. It consumes batches of trajectories and changes the policy parameters. Learning is usually accelerator-heavy and benefits from larger batches and high arithmetic intensity.

The tension is that these components prefer different execution styles. Environments prefer many independent CPU workers. Inference prefers batching. Learning prefers stable high-throughput accelerator use. Communication prefers large transfers to reduce overhead, but the control loop prefers low latency so actors do not wait too long for actions. The best architecture must reconcile these preferences.

2.3 Closed-Loop Data Generation

Reinforcement learning is a closed-loop workload. The model being trained controls the data distribution from which it learns. When the policy changes, the states visited by the agent also change. This makes systems decisions algorithmically meaningful. If actors run with old parameters for too long, their trajectories may not match the learner’s current policy. If the system uses a queue that is too deep, throughput may increase but sample freshness may decrease. If inference is centralized and delayed,

actors may idle or collect trajectories with inconsistent policy versions.

This property distinguishes RL from ordinary data-parallel supervised training. In supervised learning, stale gradients can be harmful, but the dataset itself usually remains fixed. In RL, stale policies can change both the gradient target and the distribution of experience. IMPALA and Sample Factory therefore discuss V-trace and policy lag rather than treating asynchrony as a purely systems-level optimization.

2.4 Throughput and Wall-Clock Learning

Throughput is usually measured in environment frames per second. Higher throughput means more experience per unit time. However, the final goal is not just frames per second; it is learning progress per unit wall-clock time and per unit cost. A system can produce data quickly but learn inefficiently if the algorithm becomes too off-policy or if the learner cannot consume the data. Conversely, a sample-efficient algorithm may be impractical if the system generates samples too slowly.

The three papers make this connection explicit. IMPALA reports high frame throughput and improved wall-clock performance [2]. SEED RL emphasizes speed and cost reduction [3]. Sample Factory argues that high single-machine throughput makes billion-frame experiments more accessible [11]. Their evaluations therefore consider algorithmic return alongside systems metrics such as frames per second, hardware utilization, and cost. Figure 2.1 summarizes the closed-loop pipeline that these systems optimize, and Table 2.1 lists the design questions that motivate the next section.

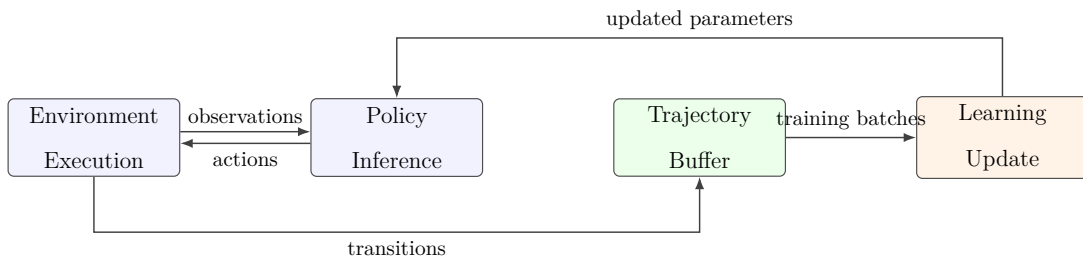


Figure 2.1: The reinforcement learning training loop as a systems pipeline. The loop is closed because the learner changes the policy that controls future data generation. This schematic is a synthesis of the workload structures discussed in [2, 3, 11].

Table 2.1: Core design questions in large-scale reinforcement learning systems.

Question	Systems interpretation
Synchronization	Barriers improve freshness but increase waiting and straggler sensitivity.
Inference placement	Placement determines CPU cost, accelerator use, communication latency, and batching.
Pipeline overlap	Overlap hides latency and keeps heterogeneous resources busy.
Policy freshness	Policy lag is a correctness consequence of asynchronous execution.
Data movement	Parameters, observations, actions, and trajectories have different size and frequency profiles.

2.5 Baseline Bottlenecks and Design Questions

The design questions in Table 2.1 organize the rest of the report. Each bottleneck below corresponds to a concrete operating-system trade-off in synchronization, placement, overlap, freshness, or data movement.

2.5.1 Synchronous Training Bottlenecks

Early parallel reinforcement learning systems often used synchronous forms of actor-critic training. Multiple workers run environments, collect steps, and then synchronize before a learner update. This design is simple and keeps the data close to the current policy, but it suffers from stragglers. If one environment is slow because of rendering variance, reset cost, or system scheduling, other workers may wait. The whole batch progresses at the speed of the slowest worker.

This problem is analogous to barrier synchronization in parallel computing. Barriers simplify reasoning but can reduce utilization. In RL, the problem is especially visible because environment step time can be irregular. Some episodes terminate and reset earlier. Some states require more simulation work. Some rendering operations take longer. When the system waits after every step or after every short rollout, the cost of these variations accumulates.

The presentation logic in `OS_pre.pptx` begins from this systems mismatch: modern RL combines simulation, inference, and learning, and these stages run at different

speeds. Poor coordination leaves CPUs or GPUs idle. The first major design question is how much synchronization can be removed without destabilizing learning.

2.5.2 Actor Inference Bottlenecks

In a distributed actor-learner architecture, actors often hold local copies of the policy. They run environments and perform inference locally. This design can scale because adding actors adds both environment stepping and inference capacity. With larger policy networks, however, CPU inference can dominate environment stepping. Increasing the number of actors then increases cost, because actors are used for simulation and for neural network computation that would often be more efficient on accelerators.

This observation motivates SEED RL. If accelerators already sit near the learner, inference may be better placed there. Centralized inference can form larger batches and improve accelerator utilization, but it changes the communication pattern: actors must send observations and wait for actions. The second major design question is where inference should run.

2.5.3 Pipeline Imbalance

Even on one machine, RL training has a pipeline structure. Rollout workers step environments, policy workers run inference, and the learner updates the model. If these stages run sequentially, the CPU waits while the GPU works and the GPU waits while the CPU works. If they run asynchronously, work can overlap. The challenge is to build queues, shared buffers, and scheduling policies that keep the pipeline full.

Sample Factory targets this problem by treating the single-node RL system as a throughput pipeline. The design works when the learner, policy workers, and rollout workers operate continuously instead of waiting for one another. The goal is familiar from operating-system design: keep resources busy while bounding latency and preserving correctness.

2.5.4 Policy Lag

Asynchrony reduces waiting but introduces policy lag. Suppose an actor collects a trajectory using behavior policy μ , while the learner currently optimizes target policy π . The difference between μ and π creates an off-policy learning problem. If the gap is small, learning may remain stable. If it is large, value targets and policy gradients can become biased or high variance.

IMPALA addresses this through V-trace, an off-policy correction method. SEED RL changes the nature of the lag because inference is centralized and policy updates take effect quickly on the learner side. Sample Factory also discusses lag and combines asynchronous execution with correction techniques. The third major design question is how to trade freshness for throughput.

2.5.5 Communication and Data Movement

Data movement is often the hidden bottleneck. In one design, actors receive parameters and send trajectories. In another, actors send observations and receive actions. In a single-machine design, components communicate through shared memory. Each choice changes the cost model.

If parameters are large and observations are small, centralized inference can reduce bandwidth. If observations are images and actions are small, sending observations at high frequency may become expensive, so streaming and batching matter. If the system is on one node, shared memory can remove serialization and network overhead. The fourth design question is what data should cross which boundary.

2.6 Workload Model and Operating-System Interface

The previous sections describe reinforcement learning as an abstract pipeline. For an operating-system-oriented discussion, the workload model needs to be more concrete.

A large-scale RL job is not a single process repeatedly calling a training function. It is a collection of cooperating execution contexts: environment workers, inference workers, replay or trajectory buffers, learners, logging processes, checkpointing processes, and sometimes evaluation workers. These contexts communicate frequently and impose different latency requirements. Environment workers need actions quickly enough to keep simulation moving. Learners prefer large, regular batches. Checkpointing must be reliable without pausing the critical path. Evaluation workers should observe a reasonably recent policy without blocking training.

This structure makes RL closer to a service pipeline than to a traditional batch job. In supervised learning, the operating system mostly provides file I/O, GPU access, memory allocation, and process scheduling. In RL, the OS also influences the timing of data generation. A scheduling delay on an environment worker changes when samples are collected. A slow queue consumer changes how stale trajectories become. A memory-copy bottleneck changes how quickly a learner can see new experience. These effects are not incidental performance details: because data are generated by the current or recent policy, timing changes the statistical input to the algorithm.

One can model the workload by treating each stage as a server with a service rate and a queue. Environment execution is measured in environment steps per second. Policy inference is measured in observations per second, with a rate that depends strongly on batch size. The learner is measured in trajectories or frames consumed per second. Communication stages are governed by serialization, memory bandwidth, network bandwidth, and RPC overhead. When one stage is slower than the previous stage, queues grow; as queues grow, latency and policy lag grow with them. The stability of the learning algorithm is therefore tied to the stability of the queueing system.

This queueing view also explains why peak throughput is sometimes misleading. A pipeline may report high actor FPS because actors continue generating samples, while the learner is already behind. In that case, the system is producing more work

than it can use. A conventional system monitor may show high CPU utilization and high sample throughput, but the learning process may suffer because samples are stale by the time they are consumed. A better monitor would show the full pipeline state: actor production rate, inference latency, learner consumption rate, queue depths, and sample age. These metrics are the RL equivalent of run-queue length, I/O wait, and service latency in an operating system.

From the OS interface perspective, large-scale RL stresses several primitives. The first is process and thread scheduling. Many environment instances may be CPU-bound, while inference and learning compete for accelerator access. The scheduler needs to handle mixed workloads without causing avoidable stalls. The second is inter-process communication. Observations and trajectories may be large tensors, so ordinary message passing can become expensive. The third is shared memory. Single-node systems such as Sample Factory rely on shared buffers to avoid repeated copies, but this requires explicit ownership discipline. The fourth is networking. Distributed systems such as IMPALA and SEED RL depend on low-overhead streams between actors and learners. The fifth is resource accounting. In a cluster, one must understand whether cost is dominated by CPU actors, accelerator learners, network traffic, or memory bandwidth.

The OS does not understand policy lag. It sees processes, buffers, sockets, and accelerators. The RL framework must build a domain-specific runtime above those primitives. That runtime decides which actor receives which policy version, when an inference batch is ready, when a trajectory is old enough to discard, and when backpressure should slow sampling. In this sense, IMPALA, SEED RL, and Sample Factory can be read as specialized operating systems for RL workloads. They implement scheduling, buffering, synchronization, and resource placement policies tailored to reinforcement learning.

This framing clarifies the main thesis of the report. The reviewed papers are not simply faster implementations of algorithms; they are alternative runtime designs. IMPALA uses a distributed actor-learner runtime with trajectory queues. SEED

RL uses remote environments and centralized inference streams. Sample Factory uses a single-node runtime with shared memory and local queues. Their differences are precisely the differences an operating-system designer would care about: where computation runs, how components communicate, how waiting is hidden, and how stale work is controlled.

3 IMPALA: Scalable Distributed Actor-Learner Training

3.1 Motivation

IMPALA was proposed to train reinforcement learning agents at large scale and across many tasks. Its motivation is that previous methods such as A3C and related actor-critic approaches were not fast enough for large multi-task settings. Training a single agent on many environments requires both high data throughput and stable learning. A purely synchronous method wastes time at synchronization points, while naive asynchronous methods can suffer from instability due to policy lag.

The paper pairs an architecture with an algorithmic correction. The architecture is the Importance Weighted Actor-Learner Architecture; the correction is V-trace [2]. The pairing matters because the architecture deliberately creates asynchrony, while V-trace makes that asynchrony usable for learning.

3.2 Architecture

The IMPALA architecture consists of a set of actors and one or more learners. Actors run environment instances. At the beginning of a trajectory, an actor retrieves recent policy parameters. It then uses its local policy to interact with the environment and collect a trajectory of observations, actions, rewards, and policy outputs. The trajectory is sent through a queue to the learner. The learner continuously consumes batches of trajectories and updates the shared policy.

This design decouples environment interaction from gradient computation. In A3C-style approaches, workers may communicate gradients or update shared param-

eters more directly. In IMPALA, actors send experience rather than gradients. The learner has access to full trajectories and can use GPU batching effectively. The queue between actors and learner absorbs timing variation among actors. Figure 3.1 illustrates this actor-queue-learner organization.

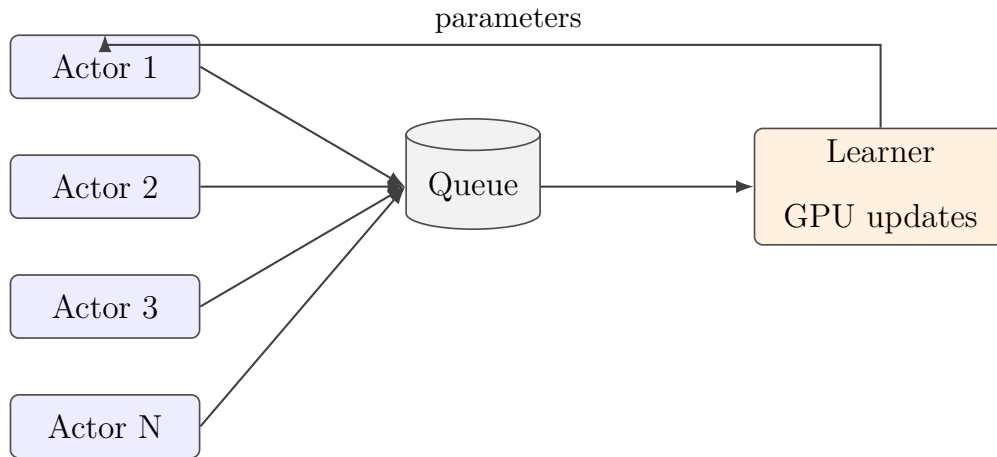


Figure 3.1: IMPALA decouples actors and learner. Actors produce trajectories asynchronously, while the learner consumes batches for GPU-based updates. The schematic is redrawn from the architecture described in [2].

3.3 Systems Interpretation

The IMPALA architecture is a producer-consumer system. Actors are producers; the learner is the consumer. A queue provides temporal decoupling. This design improves throughput because actors do not wait for learner updates after each environment step, and the learner can form large batches from many actors.

The design also improves hardware specialization. Actors can focus on environment simulation and local inference. The learner can focus on batched neural-network training. The GPU is used where it provides the most benefit: large batched updates. The architecture also supports scaling to many machines, since adding actors increases data production.

The same decoupling also introduces systems pressure. A deep queue increases trajectory staleness. A slow learner becomes a bottleneck when actors produce data faster than it can consume. Frequent parameter distribution can become expensive

at large actor counts. These details affect the learning process rather than sitting outside it.

3.4 Queueing Behavior and Backpressure

The queue between actors and learner is the central systems object in IMPALA. It is more than a buffer; it is a control surface. If the queue is short, actors may block when the learner is temporarily slower. Blocking reduces throughput, but it keeps samples fresh. If the queue is long, actors rarely block and measured FPS can rise, but the learner may train on trajectories produced many policy versions earlier. The queue mediates a trade-off between utilization and statistical freshness.

In operating-system terms, this resembles the difference between throughput-oriented buffering and latency-sensitive scheduling. A disk or network stack may use buffering to absorb bursts, but excessive buffering creates high latency. In RL, the analogue of latency is policy age. The learner is not merely receiving a late packet; it is receiving an experience sequence generated by an older decision rule. The V-trace correction is central to IMPALA because it reduces the statistical damage of delayed consumption, though it does not make queueing costless.

Backpressure prevents the system from treating unbounded buffering as harmless. If the learner falls behind, the actor side should eventually slow down, reduce rollout length, drop stale data, or add learner capacity. The original IMPALA paper focuses more on the architecture and algorithm than on a full theory of backpressure, but the need is implicit in the design. A production system would have to observe queue size and sample age continuously; otherwise, a transient learner slowdown could silently change the training distribution.

The actor side also has a scheduling problem. Actors may run different tasks, especially in the multi-task setting of DMLab-30. Some tasks step faster than others. If the learner consumes whichever trajectories arrive first, faster tasks can dominate the training stream. A multi-task system may need sampling policies above the raw

queue: per-task quotas, balanced batching, or priority rules. Here the RL runtime plays an operating-system role, deciding not just which process runs but which experience enters the learner’s batch.

Parameter distribution introduces a second queue-like delay. Actors use local policy copies, and these copies are refreshed periodically. If parameters are updated too often, distribution overhead increases. If they are updated too rarely, behavior policies become stale. The system must choose a refresh policy that balances communication cost against freshness. This refresh policy is analogous to cache coherence at the application level. Actors cache policy parameters; the learner writes new versions; the system chooses when cached copies are updated. Unlike hardware cache coherence, stale reads are allowed, but they have algorithmic consequences.

IMPALA is a clean bridge between operating systems and machine learning because its mechanisms are familiar while their consequences are not. Producers, consumers, queues, cached state, and batched processing are ordinary systems concepts. What changes is that every timing decision alters the learning problem. A queue becomes a source of off-policy data. A cached parameter copy becomes a behavior policy. IMPALA succeeds by recognizing this mismatch and designing V-trace to tolerate it.

3.5 V-trace

The key algorithmic contribution of IMPALA is V-trace, an off-policy actor-critic correction. The problem is that actors generate trajectories using a behavior policy μ , while the learner updates a target policy π . Since π may have changed after the actor copied parameters, the trajectory is off-policy by the time it reaches the learner.

V-trace uses truncated importance sampling ratios. Let

$$\rho_t = \min \left(\bar{\rho}, \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} \right), \quad c_t = \min \left(\bar{c}, \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} \right).$$

The value target is built from temporal-difference errors weighted by these clipped

ratios. Clipping limits variance while preserving a correction for policy mismatch. Intuitively, V-trace lets the learner ask how a trajectory generated by an older behavior policy should influence the current policy.

V-trace is the report’s clearest example of algorithm-system co-design. The systems goal is to reduce waiting through asynchronous actors. The algorithmic risk is that stale trajectories break on-policy learning. V-trace connects these goals.

3.6 Performance Results

The IMPALA paper reports exceptionally high data throughput for its time. A reported configuration achieves about 250,000 frames per second, around 21 billion frames per day. The system is evaluated on DMLab-30 and Atari-57, including multi-task settings where a single agent is trained across many tasks. The paper shows that high throughput can translate into improved wall-clock learning performance and positive transfer in multi-task environments.

The significance is not the frame rate alone, but the relationship among throughput, sample efficiency, and final performance. A system that produces data quickly but learns poorly would be a weak result. IMPALA shows that V-trace preserves enough data efficiency for high-throughput asynchronous training to pay off.

3.7 Strengths

IMPALA has several strengths. First, the architecture is conceptually simple. Actors generate trajectories, learners update from trajectories, and queues connect them. Second, it scales well because actors are mostly independent. Third, the use of V-trace gives a principled response to policy lag. Fourth, the system supports multi-task training by assigning actors to different tasks while sharing one policy.

These strengths explain why IMPALA became a reference point for later systems. SEED RL explicitly compares against it and modifies its inference placement. Sample Factory also discusses it as a prior high-throughput RL architecture.

3.8 Limitations

The main limitation of IMPALA is that each actor performs local inference. As the model grows, CPU inference on actors becomes expensive. Adding actors then adds cost for both environment stepping and neural network inference. Another limitation is communication overhead from parameter distribution and trajectory transfer. Finally, because the system is asynchronous, queue sizes, actor counts, and learner capacity require careful tuning.

From an operating-system perspective, IMPALA addresses synchronization bottlenecks but leaves computation placement partly unresolved. It removes waiting by decoupling acting and learning, but actors still combine simulation and inference. That remaining tension motivates SEED RL.

4 SEED RL: Accelerated Central Inference

4.1 Motivation

SEED RL begins from a critique of the conventional distributed actor-learner architecture. In systems like IMPALA, actors run environments and perform policy inference locally. That choice is reasonable for small models, but inefficient for larger neural networks. Actor machines are usually CPU-based, while neural network inference is often better suited to GPUs or TPUs. As models grow, actor-side CPU inference can dominate cost and limit scalability.

SEED RL asks a simple but powerful systems question: can inference be moved to where accelerators already are? Instead of sending parameters to actors and letting them run local inference, actors can send observations to a centralized learner-side inference service. The learner performs batched inference on accelerators and sends actions back [3].

4.2 Architecture

In SEED RL, actors are simplified. They run environments and communicate observations and actions. The learner side handles inference, trajectory accumulation, and training. Model-related computation stays near the accelerator. This design transforms actors into remote environment workers. Figure 4.1 shows the resulting communication path.

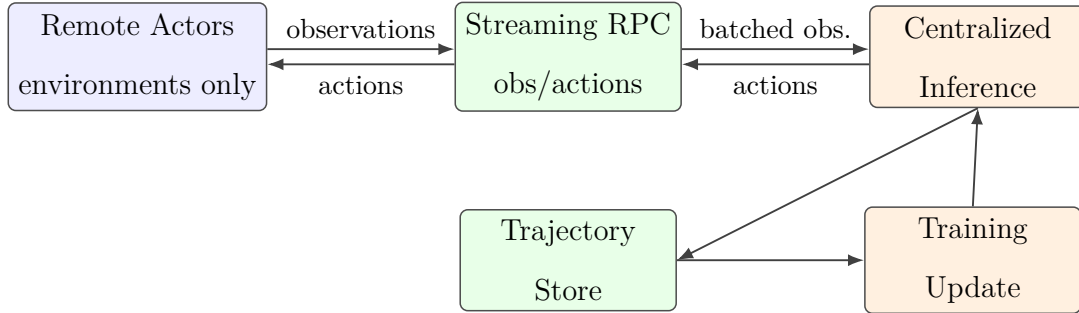


Figure 4.1: SEED RL centralizes model computation. Actors run environments and communicate observations/actions, while inference and training stay on the learner side. The schematic is redrawn from the centralized inference design in [3].

The system uses streaming remote procedure calls to reduce the overhead of frequent inference requests. The actor-learner interaction is high frequency: every environment step may require an action. Opening a separate request for every action would be too expensive. Persistent streams allow requests to be batched while avoiding much of the per-call overhead.

4.3 Computation Placement

The central design principle of SEED RL is computation placement. In IMPALA, inference is placed near the environment. In SEED RL, inference is placed near training. This shift changes the system’s resource profile.

On the positive side, centralized inference allows large accelerator batches. It removes neural-network computation from actors, so increasing model size does not require proportionally more actor CPU. It also reduces the need to distribute large model parameters to many actors. The paper notes that only observations and actions need to be sent between actors and learner-side inference.

On the negative side, actors now depend on communication latency. If an actor sends an observation and waits for an action, it can become idle. SEED RL addresses this by running multiple environments per actor and batching inference on the learner side. While one environment waits for an action, the actor may have other environment work available. The goal is to hide inference and communication

latency behind parallel environment execution.

4.4 RPC Path and Batching Semantics

SEED RL depends heavily on the request path between actors and centralized inference. At a logical level, the interaction is simple: an actor sends an observation, the inference service returns an action, and the environment continues. At systems level, the interaction occurs at very high frequency and must be implemented as a stream rather than as isolated calls. A naive RPC design would pay connection setup, serialization, scheduling, and kernel overhead for every action, potentially erasing the benefit of accelerator-side inference.

Streaming RPC changes the cost model. Persistent streams allow many observations to flow through the same communication channel. The inference service can collect pending observations from multiple actors, form an accelerator batch, run the policy network, and return actions. The resulting design resembles a high-throughput online serving system. The service must maximize batch size for accelerator efficiency while keeping response latency low enough that actors do not stall. The right point depends on environment step time and model inference time. Slow environments tolerate larger inference batches because the wait is small relative to simulation. Fast environments need lower-latency inference because every delayed action directly reduces FPS.

Batching semantics also interact with policy-version semantics. If the learner updates parameters while inference requests are in flight, nearby requests may see different policy versions. Centralized inference reduces long-lived actor-side staleness, but it does not remove time from the system. The runtime still needs to decide when newly trained parameters become visible to inference workers. Immediate updates give fresher actions but can introduce frequent synchronization between training and inference. Controlled update intervals make inference behavior more predictable at the cost of small policy-version delays. This is an application-level consistency policy.

The communication payload in SEED RL differs from IMPALA. Instead of sending model parameters to many actors, the system sends observations and actions. This is beneficial when parameters are large and observations are moderate. For high-resolution image observations or environments with many agents, however, the observation stream can become bandwidth-heavy. Compression or preprocessing may help, but it can shift work back to actors. The system designer must compare the cost of moving observations with the cost of moving parameters and computing inference locally.

Tail latency is another practical concern. Average inference latency may look acceptable while occasional slow responses stall actors. Tail latency can come from network jitter, overloaded inference batches, uneven actor scheduling, or garbage collection in high-level runtimes. In reinforcement learning, rollout workers often need actions before they can continue. Running multiple environments per actor reduces the impact of one slow response, but it does not remove the problem. A robust centralized inference service should monitor latency distributions rather than average throughput alone.

Viewed through an operating-system lens, SEED RL turns reinforcement learning into a distributed client-server system. Actors are clients that generate requests from environments. The inference service is a latency-sensitive accelerator-backed server. The learner writes the model state used by that server. The trajectory store connects online decisions to training. The separation of roles is clean, but it also makes RPC design, batching policy, and model-version consistency central to the learning system.

4.5 Policy Lag in SEED RL

SEED RL changes the nature of policy lag. In IMPALA, an actor usually uses the same copied policy for an entire trajectory. By the time the trajectory reaches the learner, the learner may have advanced several updates. In SEED RL, inference happens centrally, so actors do not keep long-lived policy copies. Policy updates can affect inference quickly. However, a trajectory may contain actions generated by

multiple nearby versions of the policy because the learner updates while the trajectory is being collected.

This creates a subtle systems-algorithm trade-off. Centralization reduces one kind of staleness but introduces a finer-grained form of policy-version mixing. The paper argues that this remains near on-policy in practice and adapts algorithms such as V-trace and R2D2 into the framework.

4.6 Cost and Throughput

SEED RL emphasizes cost as well as speed. The paper reports that for considered scenarios, cost reductions of 40% to 80% are achieved compared with prior approaches. It also demonstrates scaling to millions of frames per second in large configurations. These results matter because large-scale reinforcement learning experiments are expensive. A system that reaches the same learning result with fewer actor CPUs or better accelerator utilization directly changes research feasibility.

The cost analysis also explains why inference placement matters. If CPU actor inference is expensive, actor cost becomes a large part of the total. Moving inference to accelerators can reduce the number of actors required and make better use of hardware already allocated for training. SEED RL does not simply add compute; it changes the division of labor.

4.7 Strengths

SEED RL has a strong systems argument. It identifies actor-side inference as a bottleneck and removes it. It uses centralized batching to improve accelerator utilization. It reduces model distribution overhead. It also provides a cleaner separation between environment execution and model computation.

The architecture is especially attractive when models are large or when accelerators are efficient relative to CPU inference. It is also attractive in cloud settings where actor CPU cost is significant. From the perspective of operating systems,

SEED RL is a case study in moving computation to the resource where it is most efficient.

4.8 Limitations

Centralized inference introduces dependency on communication. If the network or RPC layer is slow, actors wait. The learner side also becomes more complex because it handles inference, training, trajectory storage, and coordination. This centralization could become a bottleneck or require careful scaling across accelerator hosts.

Another limitation is that the architecture assumes observations and actions are cheaper to communicate than parameters and local inference. This is often true, but not universally. For very large observations or environments with extremely low step latency, communication may dominate. The best inference placement remains workload-dependent.

5 Sample Factory: Single-Machine High-Throughput RL

5.1 Motivation

Sample Factory addresses a different problem from IMPALA and SEED RL. Instead of focusing primarily on large distributed systems, it asks how much throughput can be achieved on a single machine. This matters because not every research group has access to a large cluster. If careful systems design can make a single node produce hundreds of thousands of frames per second, large-scale experimentation becomes more accessible.

The paper argues that many RL systems under-utilize available hardware because the pipeline is poorly organized. Environment simulation, inference, and learning can block one another. CPU and GPU work may fail to overlap. Communication between components may add unnecessary overhead. Sample Factory responds with a high-throughput asynchronous design built around a single-node pipeline [11].

5.2 Architecture

Sample Factory divides work into rollout workers, policy workers, and a learner. Rollout workers simulate environments and collect observations. Policy workers perform batched inference on the GPU and return actions. The learner processes completed trajectories and updates the actor and critic parameters. Components communicate through FIFO queues and shared memory. Figure 5.1 presents this single-machine pipeline.

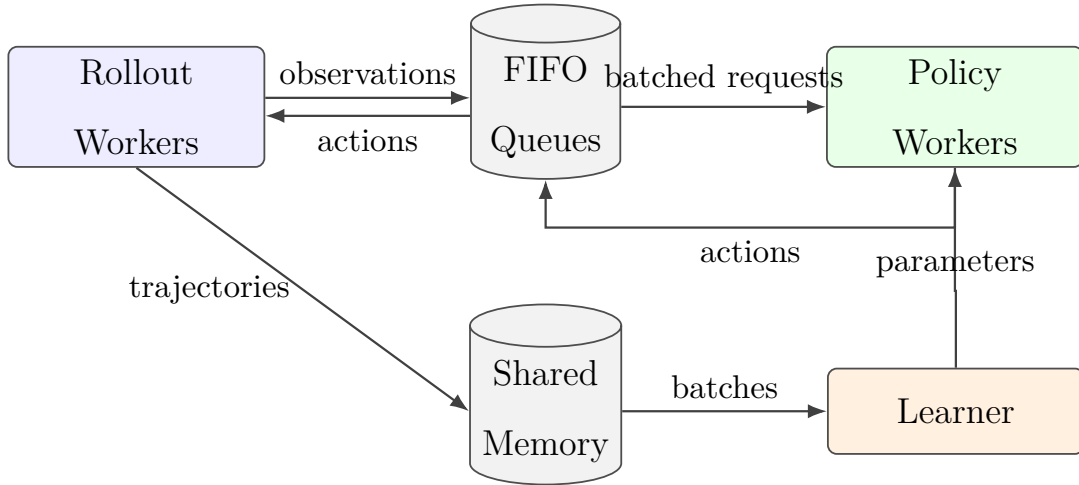


Figure 5.1: Sample Factory uses a single-machine asynchronous pipeline with rollout workers, policy workers, shared memory, and a learner. The schematic is redrawn from the architecture described in [11].

The architecture is similar in spirit to SEED RL because rollout workers do not own full policy computation. The difference is that Sample Factory implements the separation inside one machine. This lets the system use shared memory rather than network communication and reduces serialization and copying overhead.

5.3 Asynchronous Pipeline

The main systems idea in Sample Factory is continuous asynchronous execution. Rollout, inference, and learning should proceed in parallel. A rollout worker should not wait unnecessarily for the learner. A policy worker should form batches from many pending observations. The learner should consume trajectories as soon as they are ready.

This is pipeline parallelism. The throughput of the whole system is limited by the slowest stage, so the runtime must balance stages and reduce idle time. If environment stepping is slow, more rollout workers or environments may be needed. If inference is slow, larger GPU batches or more policy workers may help. If learning is slow, the learner becomes the bottleneck and may need more accelerator capacity or different batch sizes.

5.4 Double-Buffered Sampling

One distinctive mechanism in Sample Factory is double-buffered sampling. Each rollout worker manages multiple environments divided into two groups. While one group waits for actions from the policy worker, the other group continues stepping. The groups alternate. This hides inference latency and keeps the CPU busy.

The idea is widely familiar in systems design. Double buffering is used when producing and consuming data can overlap. One buffer is filled while the other is drained. In Sample Factory, the “buffers” are groups of environments and their pending observations/actions. This reduces the time rollout workers spend waiting for policy inference, as illustrated in Figure 5.2.

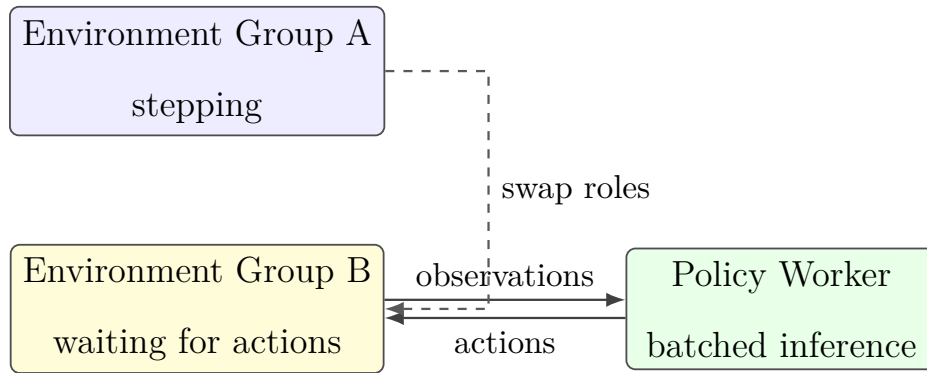


Figure 5.2: Double-buffered sampling hides inference latency by alternating environment groups, following the sampling strategy proposed in [11].

5.5 Shared Memory and Queues

Sample Factory uses shared-memory tensors and FIFO queues to connect components. This matters because trajectories can be large. Copying observations, recurrent states, logits, values, and rewards through ordinary Python objects would add substantial overhead. Shared memory lets components exchange references or indices rather than repeatedly copying data.

Queues provide coordination. They allow policy workers to batch observations from many rollout workers and allow the learner to consume trajectories when they

become available. The system resembles an application-level operating system for RL: it schedules work, coordinates buffers, and assigns resources.

5.6 Memory Ownership and Local Runtime Design

The use of shared memory in Sample Factory deserves special attention because it is one of the clearest operating-system ideas in the three main papers. Shared memory improves throughput by avoiding repeated serialization and copying. Instead of sending a large trajectory as a Python object from one process to another, components can write tensors into preallocated buffers and pass compact identifiers through queues. This approach is much closer to how high-performance systems treat data: the expensive object stays in place, and control messages describe where it is.

Shared memory, however, is not automatically safe. The runtime must define ownership. A rollout worker may own a buffer while it writes observations and rewards. A policy worker may read observation slots to produce actions. A learner may later read completed trajectories for gradient updates. If these roles overlap incorrectly, a consumer may read partially written data or a producer may overwrite data that has not yet been consumed. A shared-memory RL runtime therefore needs a protocol: which component can write each slot, when ownership transfers, and when the slot can be reused.

Queues provide part of this protocol. A queue entry can represent permission to process a buffer slot. For example, a rollout worker can enqueue an observation index for a policy worker. Once the policy worker writes an action, another queue entry can notify the rollout worker. Completed trajectory indices can then be passed to the learner. This design is similar to descriptor rings in network interfaces or block devices. The large data buffers remain fixed, while small descriptors move through queues to coordinate producer-consumer relationships.

The benefit is especially large in Python-based RL systems. Python multiprocessing can be expensive when large objects are serialized between processes. Even

if numerical computation uses optimized libraries, orchestration overhead can dominate at high environment FPS. Shared memory reduces this overhead and turns the Python layer into more of a control plane than a data plane. Sample Factory illustrates a common high-performance pattern: keep bulk data in low-overhead memory structures and use high-level code for scheduling and policy decisions.

Local runtime design also affects debugging and reproducibility. Asynchronous pipelines are harder to reason about than sequential training loops. A bug may depend on timing, queue order, or buffer reuse. For example, a missing synchronization step might appear only when the learner is unusually fast, because buffers are reused sooner. A system like Sample Factory needs strong invariants and instrumentation. Useful debugging metrics include free-buffer count, queue size, average buffer lifetime, learner lag, and policy worker batch size. These metrics are not peripheral; they reveal whether the runtime is preserving the intended dataflow.

Sample Factory also highlights a practical lesson about scale. Distributed systems often receive more attention, but local efficiency can matter more at the start. A poorly designed single-node runtime may waste CPU cycles on copies, context switches, or idle GPU time; scaling it to many machines multiplies the waste. A strong local runtime provides a better foundation for future distributed scaling. Sample Factory is therefore not just a smaller alternative to IMPALA or SEED RL; it is a reminder that the node-level operating-system problem comes before the cluster-level one.

5.7 Policy Lag and Learning Stability

As in IMPALA, asynchronous execution creates policy lag. In Sample Factory, rollout workers collect data while policy workers and the learner update parameters. The paper discusses correction methods such as V-trace and PPO clipping [12, 11]. The goal is to preserve the throughput advantage of asynchrony without losing sample efficiency.

Policy lag can be controlled through systems parameters. The rollout length, number of environments per worker, number of policy workers, and learner speed all affect how stale a sample is by the time it is used. Policy lag is a consequence of system configuration as much as an algorithmic variable.

5.8 Performance Results

Sample Factory reports very high throughput on a single machine. The paper presents a high-throughput training system that can reach approximately 130,000 environment frames per second on a multi-core node with one GPU. It evaluates performance on benchmarks including Atari, VizDoom, and DeepMind Lab. In some comparisons, Sample Factory outperforms other high-throughput RL systems on single-node setups.

The broader significance is accessibility. If a single server can support billion-frame experiments, researchers can iterate without a large cluster. This does not make distributed systems unnecessary, but it changes the baseline. Before scaling out, one should first ensure that a single node is well utilized.

5.9 Strengths and Limitations

Sample Factory’s strengths are careful local optimization, low communication overhead through shared memory, and effective overlap of CPU and GPU work. It is also practically valuable because single-machine setups are easier to reproduce than large distributed clusters.

Its limitations follow from the same design choice. A single machine has finite CPU cores, GPU memory, and memory bandwidth. The system can become learner-bound when sampling is too fast. Scaling beyond one node requires additional design for network communication and distributed coordination. Nevertheless, Sample Factory shows that much of the performance gap in RL systems can be closed by better single-node architecture.

6 Broader Related Work in Reinforcement Learning Systems

6.1 A3C and Early Asynchronous Reinforcement Learning

A3C, or Asynchronous Advantage Actor-Critic, is a direct predecessor of later actor-learner systems [9]. It uses multiple parallel actor-learners that interact with independent environment instances and update a shared neural network asynchronously. Its key insight is that parallel environment interaction can stabilize training and reduce the need for an experience replay buffer. From a systems perspective, A3C matters because it makes asynchrony part of the learning algorithm rather than a secondary implementation detail.

However, A3C also exposes the limitations of direct asynchronous updates. Workers compute gradients from their own local experience and apply updates to shared parameters. This design can be simple and lightweight, but it does not fully exploit accelerator batching. The learner workload is fragmented across workers, and GPU utilization can be poor if inference and training are not carefully batched. Later systems such as IMPALA and Sample Factory can be seen as responses to this limitation: they preserve parallel experience collection while reorganizing computation so that the expensive neural network operations can be batched and accelerated.

GA3C is another related step. It studies a GPU-based implementation of A3C and uses queues and dynamic scheduling to better leverage GPU computation [1]. Its relevance to this report is that it already treats deep reinforcement learning as

a queueing and scheduling problem. The policy network is not just a mathematical function; it is a shared service receiving requests from many environment workers. This service-oriented interpretation later becomes more explicit in SEED RL and Sample Factory.

6.2 Gorila, Ape-X, and Distributed Replay

Before IMPALA, several works explored distributed reinforcement learning through replay-based architectures. Gorila distributed DQN training across actors, learners, and parameter servers [10]. It demonstrated that deep reinforcement learning could benefit from large-scale distributed execution, but its architecture was more complex than later actor-learner designs.

Ape-X, or Distributed Prioritized Experience Replay, uses many actors to generate experience into a shared prioritized replay buffer, while a learner samples important transitions for training [5]. Compared with IMPALA, Ape-X is more replay-centered and off-policy. It decouples acting and learning through the replay buffer rather than through a FIFO trajectory queue. This changes the freshness-throughput trade-off. Replay improves data reuse and can increase sample efficiency, but it also means that data may be much older than in queue-based systems. The system must therefore decide which samples are worth replaying and how priority should be maintained.

From the operating-system viewpoint, Ape-X turns the replay buffer into a shared storage and scheduling component rather than a passive dataset. The buffer decides which experience is served to the learner, balances data from many producers, and controls the interaction between new and old samples. The analogy is close to cache replacement or priority scheduling: the buffer chooses what work receives learning attention.

6.3 R2D2 and Recurrent State Staleness

R2D2 extends distributed replay to recurrent reinforcement learning [6]. It studies the problem of training recurrent agents from distributed prioritized replay and identifies issues such as representational drift and recurrent state staleness. These issues are especially relevant here because stale data can include more than stale actions or stale policy probabilities. In recurrent agents, the hidden state used during acting may also become inconsistent with the current network representation.

This broadens the idea of policy lag. In IMPALA, policy lag is mainly the mismatch between behavior policy μ and target policy π . In R2D2, stale recurrent states add another layer: even if the action and reward sequence is available, the hidden representation associated with that sequence may no longer match the current model. The system must manage both data freshness and representation freshness.

For systems design, R2D2 reinforces the point that RL data are semantically rich. A trajectory is not just a byte array. It contains observations, actions, rewards, hidden states, priorities, and policy metadata. Queueing and storage decisions affect how meaningful those data remain when they are consumed.

6.4 RLlib and Acme as Systems Frameworks

RLlib approaches distributed reinforcement learning from the framework side [7]. It provides abstractions for distributed RL algorithms, with the goal of making irregular RL computation composable and scalable. This is relevant because many RL systems share structural patterns: rollout workers, replay buffers, learners, evaluators, and parameter synchronization. A framework must expose these components without forcing every algorithm into one rigid architecture.

Acme similarly provides a research framework for distributed reinforcement learning [4]. It separates actors, learners, datasets, replay, and environment loops in a way that supports many algorithmic variants. The existence of such frameworks

supports a central claim of this report: RL systems repeatedly need the same systems primitives. Researchers need reusable infrastructure for acting, learning, storage, logging, evaluation, and distributed coordination alongside new loss functions.

Frameworks also reveal a tension between abstraction and performance. A highly general framework may add overhead or make low-level optimizations harder. A specialized system such as Sample Factory can be faster because it controls the whole pipeline. The design question is whether to favor generality, reproducibility, and code reuse, or to optimize one workload aggressively.

6.5 DD-PPO and Synchronous Decentralized Scaling

DD-PPO, or Decentralized Distributed Proximal Policy Optimization, provides an instructive contrast to the asynchronous systems emphasized in this report [14]. It is distributed and decentralized, but synchronous: workers do not use stale computation. The paper reports near-linear scaling in resource-intensive embodied navigation environments. This shows that asynchrony is not the only path to scalable reinforcement learning.

The main point is workload dependence. If the environment is very expensive and GPU-heavy simulation dominates, synchronous decentralized training may scale well because synchronization is cheap relative to environment computation. If environment steps are cheaper and inference or learning becomes the bottleneck, asynchronous decoupling may be more attractive. The choice between synchronous and asynchronous design should follow the relative cost of stages.

6.6 EnvPool and Environment Execution Engines

EnvPool focuses on a bottleneck that the three main papers touch but do not isolate: parallel environment execution [13]. It implements a C++ thread-pool-based

executor and an asynchronous send/recv interface for reinforcement learning environments. On high-end hardware, EnvPool reports one million frames per second for Atari environment execution and three million frames per second for MuJoCo. It also maintains compatibility with common Python RL APIs.

EnvPool moves the systems lens one level lower. IMPALA, SEED RL, and Sample Factory organize actors, learners, inference, and trajectories. EnvPool asks whether the environment executor itself is too slow because of Python multiprocessing, context switching, or synchronous vectorized stepping. It separates action submission from state reception, allowing the executor to return ready batches rather than waiting for every environment instance.

This design is directly related to operating-system ideas: thread pools, asynchronous event loops, preallocated buffers, context-switch reduction, and NUMA-aware execution. It also complements Sample Factory. If Sample Factory optimizes the training pipeline and EnvPool optimizes environment execution, a future system could combine both layers.

6.7 Positioning the Three Main Papers

The broader literature clarifies the role of the three main papers. A3C introduces the value of asynchronous actor-learners. Gorila and Ape-X show the power of distributed replay. R2D2 exposes stale recurrent state as a deeper form of staleness. RLlib and Acme provide reusable distributed RL abstractions. DD-PPO demonstrates that synchronous decentralized scaling can still be effective for certain workloads. EnvPool optimizes the environment execution layer.

Within this landscape, IMPALA, SEED RL, and Sample Factory form a focused line of development around high-throughput actor-learner systems. IMPALA solves the actor-learner policy-lag problem with V-trace. SEED RL changes inference placement to reduce cost and improve accelerator use. Sample Factory shows that similar ideas can be implemented very efficiently inside one machine. The remainder of the

report compares these systems through this broader context.

7 Synthesis and Operating-System Discussion

7.1 High-Level Comparison

The three systems form a coherent progression. IMPALA introduces scalable asynchronous actor-learner training. SEED RL improves the actor-learner architecture by moving inference to centralized accelerators. Sample Factory brings similar ideas into a single-machine pipeline and pushes local utilization.

Rather than presenting the comparison as a wide table, it is clearer to read the systems as three design points. IMPALA targets distributed throughput: it keeps many actors busy and lets a learner consume asynchronous trajectories. SEED RL targets accelerator efficiency: it removes model computation from actors and makes inference a centralized batched service. Sample Factory targets local utilization: it treats one machine as a carefully scheduled pipeline in which rollout, inference, and learning overlap. These design points explain the different choices about communication, batching, and policy freshness.

7.2 Computation Placement

Computation placement is the most visible difference. IMPALA places inference on actors. SEED RL places inference on learner-side accelerators. Sample Factory separates rollout and policy inference within one machine.

The best placement depends on the relative cost of environment steps, inference, communication, and training. If environments are slow and models are small, actor-side inference may be acceptable. If models are large and accelerators are available,

centralized inference is attractive. If all components fit on one machine, shared-memory separation can provide centralization benefits without network cost.

The comparison leaves no universally optimal architecture. RL systems design is workload-sensitive. The contribution of these papers is to identify strong points in the design space and show the conditions under which they work.

7.3 Synchronization and Asynchrony

All three systems reduce synchronization. IMPALA removes frequent barriers between actors and learners. SEED RL uses streaming inference rather than synchronous parameter copying. Sample Factory overlaps rollout, inference, and learning.

Asynchrony is not free. It creates stale data, queueing delay, and more complex reasoning. The papers handle these costs through algorithmic correction, careful batching, and pipeline tuning. High-performance RL systems cannot simply maximize asynchrony; they must control it.

7.4 Communication Patterns

IMPALA communicates trajectories from actors to learners and parameters from learners to actors. This is natural for actors with local inference. SEED RL communicates observations and actions because model computation is centralized. Sample Factory communicates through shared memory and queues because components are on one node.

These patterns have different overheads. Trajectories are large but can be sent in chunks. Parameters can be very large for modern models. Observations may be image-like and frequent. Actions are usually small. Shared memory is fast but only works within a machine. A good RL system chooses the communication pattern that matches its deployment.

7.5 Relationship Between Throughput and Learning

The three papers all report high throughput, but they also care about learning performance. IMPALA shows that V-trace enables high-throughput learning without sacrificing data efficiency too severely. SEED RL shows that centralized inference can preserve sample efficiency while reducing wall-clock time and cost. Sample Factory shows that single-node throughput can translate into practical training speed.

Throughput alone is not enough. A system that generates stale or low-quality data may have high frames per second but poor final return. The relevant metric is closer to useful learning progress per wall-clock time and per dollar.

7.6 A Unified View

A unified way to view the three systems is to see each one as reducing a different kind of waiting:

- IMPALA reduces waiting caused by synchronous actor-learner barriers.
- SEED RL reduces waiting and cost caused by inefficient CPU inference and poor accelerator utilization.
- Sample Factory reduces waiting inside a single node by overlapping rollout, inference, and learning.

This “waiting reduction” view is strongly connected to operating systems. Scheduling, batching, buffering, and communication design all aim to reduce idle time while preserving correctness.

7.7 Common Design Patterns

Several design patterns recur across the reviewed systems. They abstract away from the details of any single algorithm and reveal the systems structure underneath. They also explain why the same engineering problems reappear across apparently different reinforcement learning methods.

7.7.1 Pattern 1: Decouple Data Production from Model Update

The first pattern is decoupling. Environment execution and model learning operate at different speeds, and forcing them to synchronize too frequently wastes resources. IMPALA makes this pattern explicit by letting actors produce trajectories asynchronously while the learner consumes batches [2]. Sample Factory applies the same idea inside one machine by letting rollout workers, policy workers, and the learner run as separate stages [11]. Even replay-based systems such as Ape-X and R2D2 follow the same broad principle: actors produce data into a buffer, and learners consume data according to their own schedule [5, 6].

The benefit of decoupling is throughput; the cost is freshness. Once the producer and consumer are separated, the learner may train on data generated by an older policy. A standard operating-system analogy is a queue between a producer and a consumer: the queue improves utilization and absorbs burstiness, but a long queue increases latency. In reinforcement learning, that latency has semantic meaning because older data may no longer match the current policy. Queue depth is therefore both a memory-management parameter and a learning-stability parameter.

This pattern suggests that RL systems should expose queue length, sample age, and policy version as observable metrics. A system that reports only frames per second hides whether the frames are fresh or stale. In practice, a high-throughput system should monitor both production rate and consumption delay. If the learner cannot keep up, adding more actors may improve reported FPS while reducing the

value of each sample. This is a direct interaction between a systems metric and an algorithmic metric.

7.7.2 Pattern 2: Treat Inference as a Shared Service

The second pattern treats policy inference as a service. In early actor-based systems, each worker may hold a local model and compute actions independently. This is simple, but it fragments inference into many small CPU computations. SEED RL takes the opposite position: inference is centralized near accelerators, and actors send observations to a learner-side inference service [3]. Sample Factory adopts a similar idea locally through policy workers that collect observations from rollout workers and run batched GPU inference.

The service view matters because neural network inference has strong batching effects. A GPU or TPU is inefficient when it handles one observation at a time and far more efficient when observations are batched. The inference service must therefore solve a batching problem: wait long enough to form a useful batch, but not so long that environment workers become idle. This resembles request batching in databases or RPC services, where batching improves throughput but increases tail latency.

The design also raises placement questions. If inference runs on actors, communication is cheap but computation may be inefficient. If inference runs centrally, computation is efficient but communication latency matters. SEED RL argues that for many workloads, centralized inference wins because it reduces CPU inference cost and keeps accelerators busy. The conclusion should not be read as universal. For tiny models, very large observations, or extremely fast environments, actor-side inference may still be competitive. The correct design depends on the ratio between environment step time, observation size, inference cost, and network latency.

7.7.3 Pattern 3: Hide Latency through Overlap

The third pattern is latency hiding. Sample Factory is the clearest example: rollout workers use double-buffered sampling so that one group of environments can step while another group waits for actions. This converts waiting time into useful work. SEED RL uses a related idea by running multiple environments per actor. If one environment is waiting for an inference response, the actor can still have other environment work available.

Latency hiding is a classic systems technique, but reinforcement learning makes it delicate. More overlap often means more in-flight work, which can increase policy lag. For example, if a rollout worker steps many environments while a learner updates several times, some collected transitions may be produced under older policy conditions. The benefit of overlap must be balanced against the statistical cost of staleness.

This suggests a useful design principle: apply overlap first to independent work that does not strongly increase policy delay. Overlapping CPU environment stepping with GPU inference is usually helpful; deepening a trajectory queue beyond what the learner can consume is more risky. Good RL systems need backpressure. When the learner is overloaded, actors or samplers should slow down, drop stale work, or change batch sizes rather than allow unbounded queue growth.

7.7.4 Pattern 4: Keep Data Movement Explicit

The fourth pattern is explicit data movement. RL systems move observations, actions, rewards, logits, hidden states, parameters, and trajectories. Each type of data has a different size and frequency. IMPALA moves trajectories to the learner and periodically moves parameters to actors. SEED RL keeps model parameters centralized and moves observations/actions through streaming RPC. Sample Factory uses shared memory to reduce local copies. EnvPool goes even lower and optimizes the environment execution engine to reduce Python multiprocessing overhead [13].

This pattern matters because data movement can dominate computation. A design that looks efficient algorithmically can be slow if it serializes large observations repeatedly or copies trajectory tensors between processes. Conversely, shared memory can make a single-machine design much faster, but it also increases programming complexity. The system must coordinate ownership, lifetimes, and synchronization of shared buffers. If a producer overwrites a buffer before a consumer finishes, the bug may be subtle and difficult to reproduce.

High-performance RL systems should document what crosses each boundary: process boundary, machine boundary, accelerator boundary, and language boundary. A clean architecture diagram should show not only components but also data types and update frequencies. Many performance problems become easier to diagnose once the data movement path is explicit.

7.8 Evaluation Methodology for RL Systems

A literature review of RL systems should also ask how such systems ought to be evaluated. The reviewed papers report impressive throughput numbers, but throughput is only one dimension. A strong evaluation combines system metrics, learning metrics, and cost metrics.

7.8.1 Frames per Second Is Necessary but Not Sufficient

Frames per second is the most common metric because it measures how quickly the system generates environment experience. It is easy to understand and directly related to wall-clock training time. IMPALA reports hundreds of thousands of frames per second [2]; SEED RL demonstrates million-frame-per-second scaling in large configurations [3]; Sample Factory emphasizes high single-machine throughput [11]; EnvPool reports very high raw environment execution throughput [13].

FPS can be misleading if reported alone. It may count frames that are too stale to be useful. It may increase by using more hardware even as efficiency per dollar

decreases. It also says nothing about final policy quality. A system may generate experience quickly but learn slowly because the algorithm is unstable, the batch size is poorly tuned, or the data distribution is low quality. FPS should therefore be paired with learning curves and final task performance.

A better evaluation reports wall-clock time to reach a target score. This connects system throughput to actual learning progress. For example, if one system has higher FPS but worse sample efficiency, target-score time reveals the trade-off. Cost-normalized target-score time is even better in cloud settings, because a system that is twice as fast but four times more expensive may not be preferable.

7.8.2 Utilization and Bottleneck Breakdown

Resource utilization is another essential evaluation dimension. RL systems are heterogeneous: CPU, GPU, memory bandwidth, and network links can all become bottlenecks. A good paper should show where time is spent, including environment stepping, inference, learner update, queue wait, serialization, and communication. SEED RL is strong in this regard because its motivation is tied to actor-side CPU inference cost and accelerator utilization. Sample Factory is also explicit about overlapping CPU sampling and GPU work.

Utilization numbers should be interpreted carefully. A CPU at 100% utilization is not always good if it is busy with serialization overhead rather than environment logic. A GPU at low utilization may indicate insufficient batching, but it may also indicate that environment stepping is the true bottleneck. A queue that is always full may indicate high actor throughput, but it may also indicate learner overload. Utilization should be read through a pipeline model.

A practical evaluation method is to perform ablation by removing one optimization at a time. For example, one can compare local inference with centralized inference, synchronous rollout with asynchronous rollout, shared memory with ordinary multiprocessing queues, or single-buffered rollout with double-buffered rollout. Such ablations identify which system idea actually matters for a given workload.

7.8.3 Freshness and Policy-Lag Metrics

The reviewed systems repeatedly encounter policy lag, yet many evaluations do not measure it directly. An RL systems benchmark should report the age of trajectories when they are consumed by the learner. This age can be measured in wall-clock time, learner updates, or policy versions. It should also be summarized as a distribution rather than a single average, because tail staleness may matter.

Freshness metrics would make the throughput-stability trade-off clearer. If a system improves FPS by increasing queue depth, the freshness distribution will reveal the cost. If centralized inference reduces long-lived actor policy copies, the policy-version distribution can show that the data are closer to on-policy. If double-buffering increases overlap without greatly increasing sample age, that is strong evidence that the optimization is well designed.

The same measurement helps compare algorithms. V-trace is valuable because it tolerates off-policy data in IMPALA [2]. PPO clipping can help stabilize asynchronous policy-gradient training in Sample Factory [12, 11]. Different correction methods may tolerate different levels of lag, so a system should report not only that it is asynchronous but how asynchronous it actually is.

7.8.4 Reproducibility and Configuration Sensitivity

RL systems are often sensitive to configuration. Actor count, batch size, rollout length, queue size, number of environments per worker, learner update frequency, and inference batch size can all change results. A system that performs well only under a narrow configuration may be difficult to use in practice. Frameworks such as RLlib and Acme are important partly because they encourage reproducible configurations and reusable abstractions [7, 4].

For a course report, this point matters because system design should be evaluated by robustness as well as peak performance. A robust system performs reasonably across a range of environments and models. A brittle system may require

expert tuning that hides engineering complexity. The peak numbers in the reviewed papers should therefore be read together with their assumptions: hardware type, environment type, observation size, model size, and algorithm.

7.9 Engineering Guidelines from the Literature

The literature suggests several practical guidelines for building reinforcement learning systems. First, optimize the single-node pipeline before scaling out. Sample Factory shows that poor local utilization can waste enormous performance. If rollout, inference, and learning do not overlap on one machine, adding more machines may only multiply inefficiency.

Second, place inference according to model size and observation cost. For small policies and slow environments, actor-side inference may be simple and sufficient. For larger policies or expensive CPU inference, SEED RL-style centralized inference can be more efficient. The architecture should be chosen after measuring inference latency, batch efficiency, and communication overhead.

Third, treat queues as control surfaces. Queue sizes should not be arbitrary. They control burst tolerance, throughput, latency, and sample freshness. A queue that is too small may starve the learner or inference service. A queue that is too large may hide overload and create stale samples. Backpressure is a core system feature, not an implementation detail.

Fourth, separate measurement into environment throughput, inference throughput, learner throughput, and end-to-end learning speed. A single FPS number can hide bottlenecks. The best systems papers are those that explain why the bottleneck moved after each optimization.

Fifth, design for observability. Logs should include queue depth, actor step rate, inference batch size, learner update rate, sample age, GPU utilization, CPU utilization, and network throughput. Without these metrics, debugging a distributed RL system becomes guesswork. RL failures can look like algorithmic instability even

when the root cause is a systems bottleneck.

Finally, avoid assuming that distributed is automatically better. Distributed systems introduce network latency, serialization, fault handling, synchronization complexity, and configuration burden. Sample Factory demonstrates that a well-engineered single node can be surprisingly strong. A good design should scale out only after the local pipeline is efficient and the workload genuinely needs more resources.

7.10 Operating-System Concepts in the Three Papers

7.10.1 Scheduling

Scheduling appears at multiple levels. At the cluster level, actors and learners must be placed on machines. At the node level, environment workers, policy workers, and learner threads compete for CPU cores and accelerators. At the application level, queues decide which observations form an inference batch and which trajectories form a learning batch.

IMPALA uses a coarse scheduling structure: many actors feed one or more learners. The queue absorbs variation. SEED RL schedules inference requests into accelerator batches. Sample Factory schedules rollout, inference, and learning through local queues and shared memory. In all cases, the system effectively implements a domain-specific scheduler above the operating system.

7.10.2 Synchronization

Synchronization is a central theme. Synchronous RL training is easier to reason about but can waste resources. Asynchronous systems improve throughput but require stale-data correction. The selected papers explore the boundary between systems performance and algorithmic correctness.

In IMPALA, the removal of actor-learner synchronization creates policy lag. V-trace corrects it. In SEED RL, centralized inference reduces long actor-side policy staleness but introduces continuous interaction between remote environments and learner-side inference. In Sample Factory, asynchronous local stages require careful queue and buffer design.

7.10.3 Batching

Batching is essential for accelerator utilization. GPUs and TPUs are inefficient for many tiny inference calls, but efficient for large batches. IMPALA batches trajectories for learner updates. SEED RL batches inference requests centrally. Sample Factory batches observations through policy workers.

Batching introduces a latency-throughput trade-off. Waiting longer can form a larger batch and improve accelerator efficiency, but actors may wait. Sending smaller batches reduces latency but may waste accelerator capacity. This is a classic systems trade-off, similar to disk scheduling, network packet batching, or request batching in servers.

7.10.4 Shared Memory

Sample Factory makes the clearest use of shared memory. Rollout workers, policy workers, and learner processes exchange data through shared memory tensors. This reduces copying and serialization overhead. It also requires careful coordination so that buffers are not overwritten before consumers finish.

Shared memory is powerful but local. It provides high bandwidth within one machine, but does not solve distributed communication. This is why Sample Factory is especially strong as a single-machine system, while IMPALA and SEED RL address distributed execution.

7.10.5 Remote Procedure Calls

SEED RL depends on efficient remote communication. Actors send observations and receive actions through streaming RPCs. The streaming model reduces per-request overhead and allows batching. When actors and learners are on the same machine, lower-level mechanisms such as Unix domain sockets can reduce latency and system-call overhead.

This is systems engineering below the algorithm level. The learning algorithm needs actions for observations, but performance depends on how the request-response path is implemented.

7.10.6 Load Balancing and Stragglers

Environment workers can be imbalanced. Some environments run slower; some episodes reset; some tasks are heavier. IMPALA avoids global waiting by letting actors run asynchronously. Sample Factory hides waiting with double-buffered sampling. SEED RL runs multiple environments per actor to keep CPUs useful while inference responses are pending.

These designs resemble straggler mitigation in distributed systems. Instead of waiting for all workers, the system consumes available work and keeps the critical path moving.

7.10.7 Heterogeneous Resource Management

RL systems combine CPUs, GPUs or TPUs, memory, and network. Each resource has a different performance profile. CPUs are flexible and run environments well. Accelerators are efficient for dense neural network computation. Networks add latency and bandwidth limits. Memory movement can dominate if trajectories are copied too often.

The three papers manage heterogeneity differently. IMPALA uses many CPU actors and GPU learners. SEED RL shifts inference to accelerators. Sample Factory

overlaps CPU rollout and GPU inference/training inside one machine. The common principle is to match work to the resource that performs it best.

7.11 Critical Discussion

7.11.1 Why the Three Works Are Closely Related

The three papers are not independent topics placed together. They are closely related steps in the evolution of high-throughput reinforcement learning systems. IMPALA establishes the actor-learner framework and the importance of policy-lag correction. SEED RL starts from that framework and changes inference placement to improve efficiency. Sample Factory studies similar asynchronous ideas on a single machine and emphasizes pipeline utilization.

This relationship makes the topic suitable for the course requirement of reviewing three works around one coherent theme. The common theme is system design for large-scale reinforcement learning workloads. At least one work must connect to computer systems; here, all three do, even though they appear in machine learning venues.

7.11.2 Algorithm-System Co-Design

The strongest lesson is that algorithm and system cannot be fully separated. If the system is synchronous, the algorithm can stay closer to on-policy data but runs more slowly. If the system is asynchronous, the algorithm needs correction. If inference is centralized, policy-version behavior differs from actor-local inference. If the pipeline is deeply buffered, sample freshness changes.

Systems decisions affect the statistical properties of learning. Conversely, algorithmic tolerance to stale data enables systems optimization. V-trace is the clearest example: without it, IMPALA's asynchronous architecture would be less stable; with it, the system can scale.

7.11.3 Cost and Accessibility

Large-scale RL has often been associated with expensive infrastructure. SEED RL directly addresses cost by reducing actor CPU requirements and improving accelerator utilization. Sample Factory addresses accessibility by showing that a single machine can be highly effective. These contributions matter because research progress depends on who can afford to run experiments, not only on peak performance.

From a broader systems perspective, efficiency is a democratizing force. Better scheduling and computation placement can reduce the need for brute-force hardware scaling.

7.11.4 Remaining Challenges

Several challenges remain open. First, centralized inference can become a bottleneck for very large numbers of actors or very large observations. Second, policy lag is still difficult to reason about in complex multi-agent or recurrent settings. Third, single-machine designs can hit memory bandwidth or learner bottlenecks when environment execution becomes too fast. Fourth, fault tolerance is less emphasized in these papers than throughput, but large distributed systems must handle machine failures and partial slowdowns.

Another challenge is portability. A system optimized for Atari may not perform the same way for robotics simulators, language-based environments, or multi-agent games. The best architecture depends on environment step time, observation size, model size, and hardware availability.

7.11.5 Possible Future Directions

Future RL systems may combine the strengths of the three works. A large system could use SEED RL-style centralized inference across accelerator hosts, Sample Factory-style single-node pipelines inside each host, and IMPALA-style off-policy correction for asynchronous trajectories. It could also use adaptive scheduling: when

inference becomes the bottleneck, increase batching or accelerator allocation; when environment stepping becomes the bottleneck, increase rollout workers; when learner queues become too deep, reduce sampling rate or adjust update frequency.

Another direction is to make policy freshness a first-class scheduling metric. Instead of only maximizing frames per second, the system could optimize useful fresh frames per second. This would connect queue management and learning stability more directly.

8 Conclusion

This thesis-style report reviewed IMPALA, SEED RL, and Sample Factory as three representative systems for large-scale reinforcement learning workloads. Following the logic of the course presentation, it argued that modern RL is difficult not only because of sparse rewards and exploration, but also because training requires massive interaction with environments. When RL is viewed as a systems workload, architecture becomes central.

IMPALA decouples actors and learners to reduce synchronization bottlenecks. Its V-trace algorithm corrects the policy lag created by asynchronous execution. SEED RL moves inference to centralized accelerators, reducing actor-side CPU cost and improving utilization through batched inference and streaming RPC. Sample Factory optimizes the single-machine pipeline with rollout workers, policy workers, a learner, shared memory, queues, and double-buffered sampling.

The common lesson is that high-performance RL is algorithm-system co-design. Better systems do more than add hardware. They reduce waiting, place computation near suitable resources, avoid unnecessary data movement, and choose synchronization points carefully. At the same time, learning algorithms must tolerate the staleness and asynchrony that make the system efficient.

For an Advanced Operating System course, these papers show how OS ideas appear inside modern machine-learning workloads. Scheduling, synchronization, batching, memory sharing, RPC, load balancing, and heterogeneous resource management are not background details. They shape which reinforcement learning experiments are practical.

Bibliography

- [1] Mohammad Babaeizadeh, Iuri Frosio, Stephen Tyree, Jason Clemons, and Jan Kautz. GA3C: GPU-based A3C for Deep Reinforcement Learning. arXiv:1611.06256, 2016.
- [2] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [3] Lasse Espeholt, Raphaël Marinier, Piotr Stanczyk, Ke Wang, and Marcin Michalski. SEED RL: Scalable and Efficient Deep-RL with Accelerated Central Inference. In *International Conference on Learning Representations*, 2020.
- [4] Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, Sarah Henderson, Alex Novikov, Sergio Gomez Colmenarejo, Serkan Cabi, Caglar Gulcehre, Tom Le Paine, Andrew Cowie, Ziyu Wang, Bilal Piot, and Nando de Freitas. Acme: A Research Framework for Distributed Reinforcement Learning. arXiv:2006.00979, 2020.
- [5] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed Prioritized Experience Replay. In *International Conference on Learning Representations*, 2018.
- [6] Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent Experience Replay in Distributed Reinforcement Learning. In *International Conference on Learning Representations*, 2019.

- [7] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. RLLib: Abstractions for Distributed Reinforcement Learning. In *International Conference on Machine Learning*, 2018.
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 2015.
- [9] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In *International Conference on Machine Learning*, 2016.
- [10] Arun Nair, Praveen Srinivasan, Sam Blackwell, Cagdas Alcicek, Rory Fearon, Alessandro De Maria, Vedavyas Panneershelvam, Mustafa Suleyman, Charles Beattie, Stig Petersen, Shane Legg, Volodymyr Mnih, Koray Kavukcuoglu, and David Silver. Massively Parallel Methods for Deep Reinforcement Learning. In *ICML Deep Learning Workshop*, 2015.
- [11] Aleksei Petrenko, Zhehui Huang, Tushar Kumar, Gaurav Sukhatme, and Vladlen Koltun. Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- [12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
- [13] Jiayi Weng, Min Lin, Shengyi Huang, Bo Liu, Denys Makoviichuk, Viktor Makoviychuk, Zichen Liu, Yufan Song, Ting Luo, Yukun Jiang, Zhongwen Xu,

- and Shuicheng Yan. EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine. In *Advances in Neural Information Processing Systems*, 2022.
- [14] Erik Wijmans, Abhishek Kadian, Ari Morcos, Stefan Lee, Irfan Essa, Devi Parikh, Manolis Savva, and Dhruv Batra. DD-PPO: Learning Near-Perfect PointGoal Navigators from 2.5 Billion Frames. In *International Conference on Learning Representations*, 2020.